

✿ Ansible Roles:

Overview

- In the real world, play might be long and complex, with many included or imported files, and with tasks and handlers to manage various situations.
- Copying all that code into another playbook might be nontrivial work.
- Ansible roles provide a way for you to make it easier to reuse Ansible code generically.
- You can package, in a standardized directory structure, all the tasks, variables, files, templates, and other resources needed to provision infrastructure or deploy applications.
- Copy that role from project to project simply by copying the directory. You can then simply call that role from a play to execute it.
- A well-written role will allow you to pass variables to the role from the playbook that adjust its behavior, setting all the site-specific hostnames, IP addresses, user names, secrets, or other locally-specific details you need.
- For example, a role to deploy a database server might have been written to support variables which set the hostname, database admin user and password, and other parameters that need customization for your installation.
- The author of the role can also ensure that reasonable default values are set for those variables if you choose not to set them in the play.

✿ What Are Ansible Roles?

- Roles are a structured way of grouping together various functionalities and making it easier to reuse and share common setup tasks.
- Roles are a structured way to organize playbooks into reusable components.
- Instead of writing all tasks, handlers, variables, and templates in one big playbook, you split them into roles.
- Each role has a standard directory layout so Ansible knows where to find things.

✿ Standard Role Subdirectories:

Subdirectory	Purpose
tasks/	Contains the main list of tasks to execute. Usually starts with main.yml.
handlers/	Defines handlers (special tasks triggered by notify, e.g., restart a service).
files/	Stores static files that can be copied to managed hosts.
templates/	Holds Jinja2 templates (.j2) for dynamic configuration files.
vars/	Contains variables with high precedence (overrides defaults).
defaults/	Contains default variables (lowest precedence, can be overridden easily).
meta/	Defines role metadata, including dependencies on other roles.
tests/	Provides test playbooks and inventory for validating the role.
library/	Custom Ansible modules specific to the role.
module_utils/	Helper code for custom modules.
lookup_plugins/	Custom lookup plugins for the role.

Ansible roles have the following benefits:

- Roles group content, allowing easy sharing of code with others.
- Roles can be written that define the essential elements of a system type: web server, database server, Git repository, or other purpose
- Roles make larger projects more manageable.
- Roles can be developed in parallel by different administrators.

*In addition to writing, using, reusing, and sharing your own roles, you can get roles from other sources. Some roles are included as part of Red Hat Enterprise Linux, in the **rhel-system-roles package**. You can also get numerous community-supported roles from the Ansible Galaxy website.*

✖ USING ANSIBLE ROLES IN A PLAYBOOK:

Using roles in a playbook is straightforward.

How Roles Behave When Used in a Playbook

1. Basic Role Usage in a Play

A simple play calling two roles:

- hosts: remote.example.com

roles:

- role1

- role2

When you list roles like this, Ansible automatically imports:

1. Role **tasks**
2. Role **handlers**
3. Role **vars**
4. Role **defaults**
5. Role **dependencies** (from meta/main.yml)

All of this is inserted into the play **before** any normal tasks you define.

2. How Role File Lookup Works

Inside a role:

- copy: looks in roles/<role>/files/
- template: looks in roles/<role>/templates/
- include_tasks: or import_tasks: looks in roles/<role>/tasks/

No need for absolute paths — Ansible resolves them automatically.

3. Setting Role Variables Inline (Role Parameters)

When you call a role inside the roles: section of a play, you can pass variables directly to that role at the moment it is invoked.

```
``yaml
```

```
roles:
```

```
- role: role2
```

```
  var1: val1
```

```
  var2: val2
```

```
...
```

These variables (var1, var2,) are called role parameters.

- They are not inventory variables.
- They are not defaults.
- They are not vars inside the role.
- They are values injected into the role at call time.

Role parameters have extremely high precedence.

They override:

- defaults
- vars
- inventory vars
- play vars

This is why you must **never reuse the same variable name elsewhere** in the play.

4. Execution Order Inside a Play

- For each play in a playbook, tasks execute as ordered in the tasks list. After all tasks execute, any notified handlers are executed.
- When a role is added to a play, role tasks are added to the beginning of the tasks list. If a second role is included in a play, its tasks list is added after the first role.
- if you add a role after some tasks in role section, the role will still run first.

```
- hosts: web
```

```
tasks:
```

```
- debug:
```

```
  msg: "task 1"
```

```
roles:
```

```
- myrole
```

The role still runs BEFORE your tasks.

Because the role section is always processed before the section, no matter where you write it in the YAML file.

- handlers are added to plays in the same manner that role tasks are added to plays. Each play defines a handlers list. Role handlers are added to the handlers list first, followed by any handlers defined in the handlers' section of the play.

When Ansible processes a play, it builds a handler list in this exact order:

1. Handlers from roles (in the order roles are listed)
2. Handlers defined in the **play's handlers:** section

So if you write

```
...
```

```
roles:
```

```
- role1
```

```
- role2
```

```
handlers:
```

```
- name: my_play_handler
```

```
  debug:
```

```
    msg: "handler from play"
```

```
...
```

Ansible internally builds:

```
...
```

```
handlers:
```

```
- handlers from role1
```

```
- handlers from role2
```

```
- my_play_handler
```

```
...
```

Why Ansible Does This?

Roles are meant to be self-contained units:

- They install packages
- Configure services
- Manage files
- Restart services

So, their handlers must be available before your tasks run, because your tasks might notify them.

If play handlers were loaded first, roles wouldn't be able to rely on their own handlers.

✓ **Handlers from roles are available to:**

- role tasks
- play tasks
- pre_tasks
- post_tasks

✓ **Play handlers come last**

This allows you to override a role's handler by defining a handler with the same name in the play.

5. Adding Roles as Tasks (Dynamic vs Static)

You can include a role **inside the tasks list**:

Dynamic include (runtime decision)

- name: include role dynamically

include_role:

name: role2

Static import (parsed at playbook load time)

- name: import role statically

import_role:

name: role2

When to use which?

Method	Behavior	Use Case
import_role	Static, parsed early	When you want predictable task ordering
include_role	Dynamic, runtime	When conditions decide whether the role runs

This mirrors the same logic as import_tasks vs include_tasks.

✿ In certain scenarios, it may be necessary to execute some play tasks before the roles. To support such scenarios, plays can be configured with a **pre_tasks** section. Any task listed in this section executes before any roles are executed. If any of these tasks notify a handler, those handler tasks execute before the roles or normal tasks. Plays also support a **post_tasks** keyword. These tasks execute after the play's normal tasks, and any handlers they notify, are run.

Why `pre_tasks` and `post_tasks` Exist

By default:

- **roles run before tasks**
- **handlers run after tasks**

But sometimes you need:

- tasks that run **before roles**
- tasks that run **after everything else**

That's exactly what `pre_tasks` and `post_tasks` are for.

✿ 1. `pre_tasks` — Tasks That Run *Before* Roles

`pre_tasks` are executed **first**, before:

- role defaults
- role vars
- role tasks
- normal tasks

✓ When do you use `pre_tasks`?

- Preparing the system before a role runs
- Installing something required by a role
- Gathering custom facts
- Validating conditions
- Stopping a service before a role configures it

✓ Handler behavior

If a `pre_task` notifies a handler:

That handler runs immediately after all `pre_tasks` finish — BEFORE roles start.

This is unusual but sometimes necessary (e.g., stopping a service before a role configures it).

✿ 2. `post_tasks` — Tasks That Run *After* Everything Else

`post_tasks` run **after**:

- role tasks
- normal tasks

- handlers triggered by those tasks

✓ When do you use `post_tasks`?

- Cleanup
- Final verification
- Sending notifications
- Environment-specific tweaks
- Final reporting

✓ Handler behavior

If a `post_task` notifies a handler:

That handler runs **after all `post_tasks` finish** — the final step of the play.

Full Execution Order (Exact Sequence)

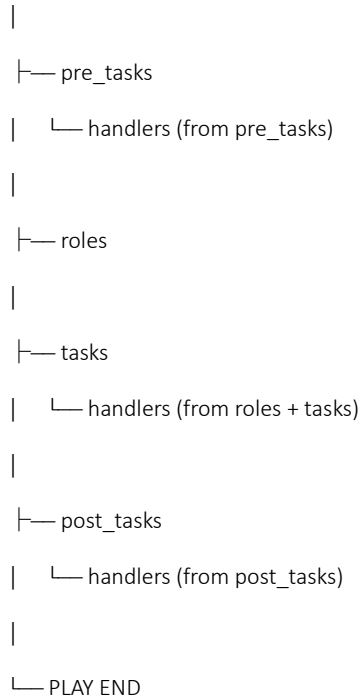
Here is the real, internal order Ansible follows:

1. **`pre_tasks`**
2. handlers notified by `pre_tasks`
3. **`roles`**
4. **`tasks`**
5. handlers notified by roles or tasks
6. **`post_tasks`**
7. handlers notified by `post_tasks`

This is the complete lifecycle of a play.

🌿 Visual Diagram

PLAY START



🌿 DEFINING VARIABLES AND DEFAULTS

- Role variables are defined by creating a `vars/main.yml` file with key: value pairs in the role directory hierarchy.
- They are referenced in the role YAML file like any other `variable: {{ VAR_NAME }}`. These variables have a high precedence and cannot be overridden by inventory variables.
- The intent of these variables is that they are used by the internal functioning of the role.
- Default variables allow default values to be set for variables that can be used in a play to configure the role or customize its behavior. They are defined by creating a `defaults/main.yml` file with key: value pairs in the role directory hierarchy.
- Default variables have the lowest precedence of any variables available. They can be easily overridden by any other variable, including inventory variables. These variables are intended to provide the person writing a play that uses the role with a way to customize or control exactly what it is going to do. They can be used to provide information to the role that it needs to configure or deploy something properly.
- Define a specific variable in either `vars/main.yml` or `defaults/main.yml`, but not in both places. Default variables should be used when it is intended that their values will be overridden

✖ Understanding Role Variable Precedence

🔒 1. vars/main.yml — High-precedence, internal role variables

These variables:

- Have **very high precedence**
- **Cannot be overridden** by inventory, group_vars, host_vars, or playbook vars
- Are intended for **internal logic** inside the role
- Should be used only when the role *must* enforce a specific value

When to use vars/main.yml

Use it for:

- Internal constants
- Values that should never be changed by the user
- Role-internal paths, filenames, or logic switches

Example

```
# roles/myrole/vars/main.yml
```

```
internal_config_path: "/opt/myrole/config"
```

This value is locked. The user cannot override it.

⚙ 2. defaults/main.yml — Low-precedence, user-customizable variables

These variables:

- Have the **lowest precedence** in all of Ansible
- Are **meant to be overridden** by inventory, playbooks, extra vars, etc.
- Provide **safe defaults** for role behavior
- Allow the user to configure the role without modifying it

When to use defaults/main.yml

Use it for:

- Options the user may want to change
- Tunable parameters
- Deployment-specific values
- Anything that should be flexible

Example

```
# roles/myrole/defaults/main.yml
```

```
service_port: 8080
```

```
enable_ssl: false
```

The user can override these anywhere:

```
- hosts: web
```

```
roles:
```

```
- role: myrole
```

```
vars:
```

```
    service_port: 9090
```

Why You Never Duplicate Variables in Both?

If a variable exists in both vars/ and defaults/, the one in vars/ will always win — making the default useless and confusing.

So the rule is:

If the value should be override-able → put it in defaults/.

If the value must never be overridden → put it in vars/.

Practical Guidance for Real-World Role Design

Use vars/ for:

- Internal constants
- Hardcoded paths used only by the role
- Logic switches the user should not touch

Use defaults/ for:

- Ports
- Package names
- Feature toggles
- File destinations
- Configuration options
- Anything environment-specific

Never put secrets in either

Secrets belong in:

- Inventory
- Vault files
- External secret stores

Roles should only *consume* them via variables.

Roles should not have site-specific data in them. They definitely should not contain any secrets like passwords or private keys. This is because roles are supposed to be generic, reusable, and freely shareable. Site-specific details should not be hard coded into them. Secrets should be provided to the role through other means. This is one reason you might want to set role variables when calling a role. Role variables set in the play could provide the secret, or point to an Ansible Vault-encrypted file containing the secret.

Core Idea

Roles must stay generic, reusable, and shareable.

Anything tied to a specific environment especially secrets, must *never* live inside a role.

Why Roles Must Not Contain Site-Specific Data

Roles are meant to be:

- **Reusable** across multiple environments (dev, QA, prod)
- **Shareable** with teams or the community (Galaxy, Git repos)
- **Composable** inside larger automation frameworks

If you hard-code environment-specific values (paths, hostnames, usernames, ports), the role becomes tightly coupled to one site and loses its portability.

Why Secrets Must Never Be Inside Roles

Secrets include:

- Passwords
- Private keys
- API tokens
- Certificates

- Vault credentials

Embedding these inside a role is dangerous because:

- Roles are often version-controlled → secrets leak into Git
 - Roles may be shared → secrets leak to others
 - Roles may be reused in multiple environments → wrong secrets get applied
 - Secrets rotate → role becomes outdated and brittle
-

✓ Correct Ways to Provide Secrets to Roles

You provide secrets **from outside the role**, using one of these patterns:

1. Role Variables Passed from the Playbook

```
- hosts: web

roles:

- role: nginx

vars:

    ssl_key_path: "/etc/ssl/private/site.key"
```

2. Inventory Variables

Group vars or host vars:

group_vars/production.yml

host_vars/web01.yml

3. Ansible Vault

Encrypt the sensitive data:

```
ansible-vault encrypt group_vars/production/secrets.yml
```

Then load it:

```
vars_files:

- "group_vars/production/secrets.yml"
```

4. External Secret Stores

(Recommended for production)

- HashiCorp Vault
- AWS Secrets Manager

- Azure Key Vault
- GCP Secret Manager

Roles simply *consume* the secret via variables.

Multi-role project skeleton

- Create Local Repo.
- Create a sample webserver.
- Create Users.

Use BM Command to create role like repo_setup,nginx,users

ansible-galaxy init role_name

Project Layout

ansible-project/

└─ site.yml # Main orchestration playbook

└─ inventory/hosts # Inventory file

└─ roles/

 └─ repo_setup/

 └─ tasks/main.yml

 └─ vars/main.yml

 └─ templates/repo.j2

 └─ defaults/main.yml

 └─ nginx/

 └─ tasks/main.yml

 └─ handlers/main.yml

 └─ templates/nginx.conf.j2

 └─ defaults/main.yml

└─ users/

 └─ tasks/main.yml

 └─ defaults/main.yml

Main Orchestration Playbook (site.yml)

- name: Prepare all servers

hosts: webservers

become: yes

roles:

- repo_setup

- users

- name: Configure web servers

hosts: webservers

become: yes

roles:

- nginx

Role Examples

roles/repo_setup/templates/repo.j2

/etc/yum.repos.d/local-AppStream

[local-AppStream]

name=Local AppStream

baseurl=file:///myrepo/AppStream

enabled=1

gpgcheck=0

/etc/yum.repos.d/local-BaseOS.repo

[local-BaseOS]

name=Local BaseOS

baseurl=file:///myrepo/BaseOS

enabled=1

gpgcheck=0

roles/repo_setup/tasks/main.yml

```
- name: Configure local repo

template:

  src: repo.j2

  dest: /etc/yum.repos.d/local.repo
```

roles/nginx/templates/nginx.conf.j2

```
server {

    listen    80;

    server_name localhost;


    location / {

        root  /newsite;

        index index.html index.html;

    }

}
```

roles/nginx/tasks/main.yml

```
- name: Install NGINX

package:

  name: nginx

  state: present

- name: Deploy nginx.conf

template:

  src: nginx.conf.j2

  dest: /etc/nginx/conf.d/default.conf

notify: Restart nginx
```

roles/nginx/handlers/main.yml

- name: Restart nginx

service:

name: nginx

state: restarted

enabled: yes

roles/users/tasks/main.yml

- name: Create lab users

user:

name: "{{ item.name }}"

shell: "{{ item.shell }}"

state: present

loop: "{{ lab_users }}"

roles/users/defaults/main.yml

lab_users:

- { name: "devops", shell: "/bin/bash" }

- { name: "tester", shell: "/bin/bash" }

Why This Skeleton Works

- **Modularity:** Each role handles one responsibility (repos, nginx, users).
- **Reusability:** You can reuse repo_setup or users in other projects.
- **Scalability:** Add more roles (e.g., db, monitoring) without bloating the main playbook.
- **Clarity:** site.yml just orchestrates roles, keeping it readable.